Regression

1. จงสร้างโมเดลที่สามารถสร้างโมเดลเรียนรู้เชิงเลขและประเมินผลสัมประสิทธิ์การตัดสินใจระหว่างต้นแปรต้นต่อตัวแปรตามที่สูงที่สุด

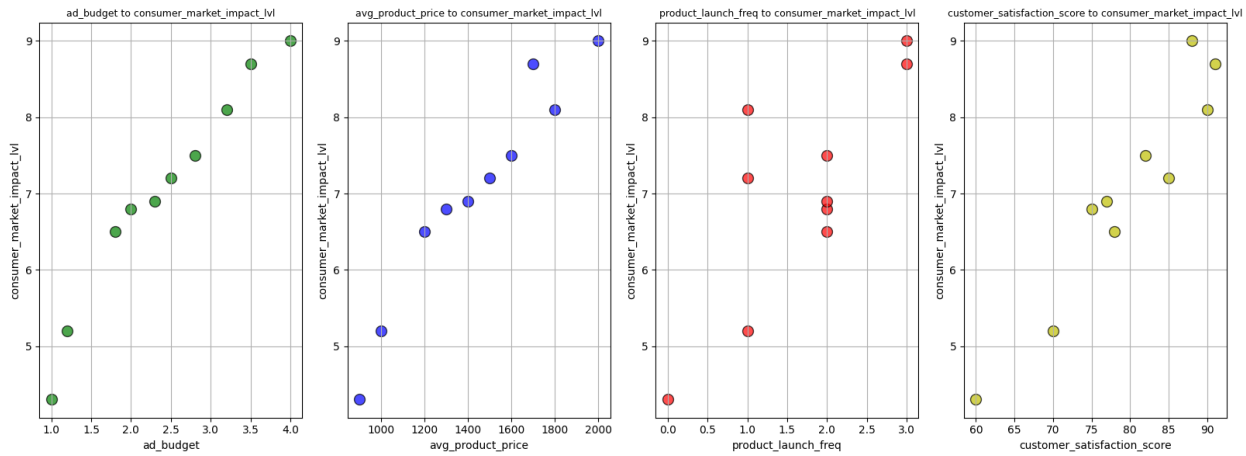
เอา dataset ที่อาจารย์ให้มาทำเป็น Dataframe ของ pandas

```
data = {
    "avg_product_price": avg_product_price,
    "ad_budget": ad_budget,
    "product_launch_freq": product_launch_freq,
    "customer_satisfaction_score": customer_satisfaction_score,
    "consumer_market_impact_lvl": consumer_market_impact_lvl,
}

df = pd.DataFrame(data)

df.head()
```

ทำ X, y เพื่อเอาไปเทรนโดยใช้ทุก feature มาทำเป็น X ส่วน y ใช้ Consumer Market Impact Level feature

```
from sklearn.preprocessing import StandardScaler

selected_features = [
    "avg_product_price",
    "ad_budget",
    "product_launch_freq",
    "customer_satisfaction_score",
]
X = df[selected_features].values
y = df["consumer_market_impact_lvl"].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
✓ 0.0s
```

Model ที่ใช้จะใช้เป็น Multiple Linear Regression เนื่องจากข้อมูล dataset ที่ได้มีความเป็น linear จากรูปด้านล่าง



เอาข้อมูลมาเทรนด้วย Multiple Linear Regression จะได้ R2 เท่ากับ 0.995 และ mse เป็น 0.0089

```
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X_scaled, y)
y_pred_sklearn = lin_reg.predict(X_scaled)

r2_sklearn = r2_score(y, y_pred_sklearn)
mse_sklearn = mean_squared_error(y, y_pred_sklearn)
print(f"Scikit-learn R^2: {r2_sklearn}")
print(f"Scikit-learn MSE: {mse_sklearn}")
```

✓ 0.0s

```
Scikit-learn R^2: 0.9953538016603131
Scikit-learn MSE: 0.008928134729542379
```

2. ระดับผลกระทบต่อตลาดผู้บริโภคที่ได้ 7.88

```python
test_dataset = pd.DataFrame(
    {
        "avg_product_price": [900],
        "ad_budget": [0.5],
        "product_launch_freq": [6],
        "customer_satisfaction_score": [90],
    }
)

X_new = test_dataset[selected_features].values
X_new_scaled = scaler.transform(X_new)
y_new_pred_sklearn = lin_reg.predict(X_new_scaled)
print(f"Scikit-learn Prediction for New Data: {y_new_pred_sklearn[0]}")
```

✓ 0.0s

```
Scikit-learn Prediction for New Data: 7.886670064468782
```

Classification

1. จงประมาณการโมเดลเรียนรู้ที่สามารถจำแนกประเภทพลังงานตามเกณฑ์ที่กำหนดและให้ความแม่นยำในการทำนายที่สูง

   เอาข้อมูลที่ได้มาทำ label โดยให้ข้อมูลพลังงานต่ำกว่า 500 เป็น class 0 ข้อมูลอยู่ระหว่าง 500 กับ 600 เป็น class 1 และมากกว่า 600 เป็น class 2 แล้วเอาข้อมูลมากแบ่ง test size 0.2 แล้วก็ทำเป็น dataloader ใช้ batchsize 16

```python
import pandas as pd


df = pd.read_csv(
    "/Users/kitton/.cache/kagglehub/datasets/sajjadtahmasebi/wind-turbine-dataset/versions/1/wind_turbine_dataset.csv"
)

df = df[pd.to_numeric(df["energy_output"], errors="coerce").notnull()]

df["energy_output"] = df["energy_output"].apply(
    lambda x: 0 if x ≤ 500 else (1 if x ≤ 600 else 2)
)
```

0.0s

```python
import torch.nn as nn
import torch
import pandas as pd
from sklearn.model_selection import train_test_split

# from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset
from tqdm import tqdm
import matplotlib.pyplot as plt
```

0.0s

```python
df.dropna(inplace=True)  # Drop rows with NaN
X = df.drop(["energy_output", "turbine_status"], axis=1)[:-1].values
y = df["energy_output"][:-1].values

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.long)
X_valid_tensor = torch.tensor(X_valid, dtype=torch.float32)
y_valid_tensor = torch.tensor(y_valid, dtype=torch.long)

batch_size = 16
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

test_dataset = TensorDataset(X_valid_tensor, y_valid_tensor)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

0.0s

เลือกใช้ MLP เนื่องเป็น Model ที่มีการแบ่งเป็น Layer เป็น Hidden Layer ทำให้ผมคิดว่า มันจะทำให้เรียนรู้ข้อมูลได้ดี โดยผมได้มีการใส่ BatchNorm เข้าไปด้วย เพื่อทำ standardization ในแต่ละ neuron และ dropout สุ่มปิดบาง node เพื่อให้ model เรียนรู้ได้ดียิ่งขึ้น

```python
class MLP(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(MLP, self).__init__()
        self.layers = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 32),
            nn.BatchNorm1d(32),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(32, output_dim),
        )

    def forward(self, x):
        return self.layers(x)
```

✓ 0.0s

```python
def train(
    model, train_loader, criterion, optimizer, epochs=50, scheduler=None, lambda_lr=No
):
    train_losses = []
    train_accuracies = []
    val_losses = []
    val_accuracies = []

    for epoch in range(epochs):
        model.train()
        train_acc = 0
        train_loss = 0

        for X_batch, y_batch in tqdm(
            train_loader, desc=f"Epoch {epoch + 1}/{epochs} - Training"
        ):
            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)

            if lambda_lr:
                l2_norm = sum(p.pow(2.0).sum() for p in model.parameters())
                loss = loss + lambda_lr * l2_norm

            loss.backward()
            optimizer.step()

            if scheduler:
                scheduler.step()

            train_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            train_acc += (predicted == y_batch).sum().item()

        avg_train_loss = train_loss / len(train_dataset)
        avg_train_acc = train_acc / len(train_dataset)

        train_losses.append(avg_train_loss)
        train_accuracies.append(avg_train_acc)

        model.eval()
        val_acc = 0
        val_loss = 0
        with torch.no_grad():
            for X_batch, y_batch in tqdm(
                test_loader, desc=f"Epoch {epoch + 1}/{epochs} - Validation"
            ):
                outputs = model(X_batch)
                loss = criterion(outputs, y_batch)
                val_loss += loss.item()
                _, predicted = torch.max(outputs, 1)
                val_acc += (predicted == y_batch).sum().item()

        avg_val_loss = val_loss / len(test_dataset)
        avg_val_acc = val_acc / len(test_dataset)
        val_losses.append(avg_val_loss)
        val_accuracies.append(avg_val_acc)

        print(
            f"Epoch {epoch + 1}/{epochs} - "
            f"Train Loss: {avg_train_loss:.4f}, Train Acc: {avg_train_acc:.4f}, "
            f"Val Loss: {avg_val_loss:.4f}, Val Acc: {avg_val_acc:.4f}"
        )

    return train_losses, train_accuracies, val_losses, val_accuracies
```

✓ 0.0s

Optimizer ใช้เป็น Adam

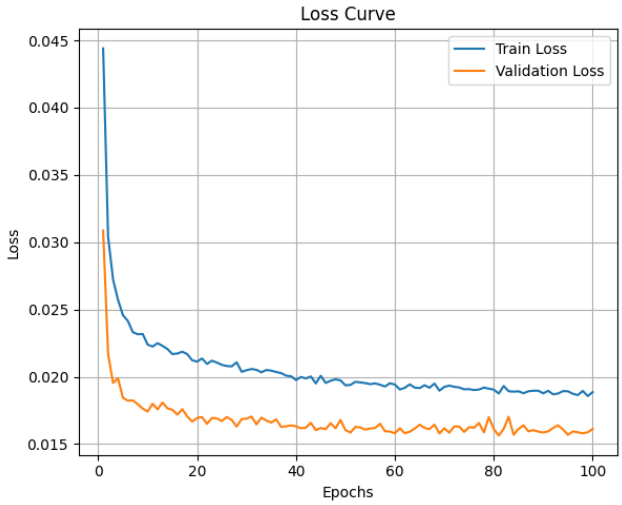
LR -> 0.0001

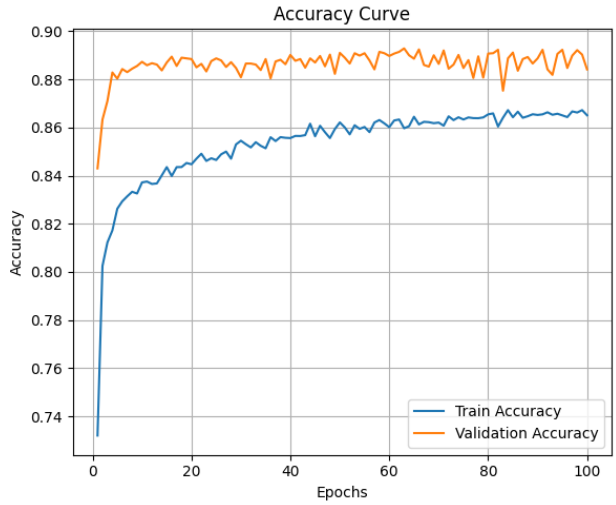
```python
model = MLP(input_dim=X_train_tensor.shape[1], output_dim=3)

criterion = nn.CrossEntropyLoss()


optimizer, scheduler = get_optimizer_scheduler(
    "Adam", model.parameters(), lr=0.0001, scheduler_name=None
)
epochs = 100

train_losses, train_accuracies, val_losses, val_accuracies = train(
    model, train_loader, criterion, optimizer, epochs=epochs, scheduler=scheduler
)
```

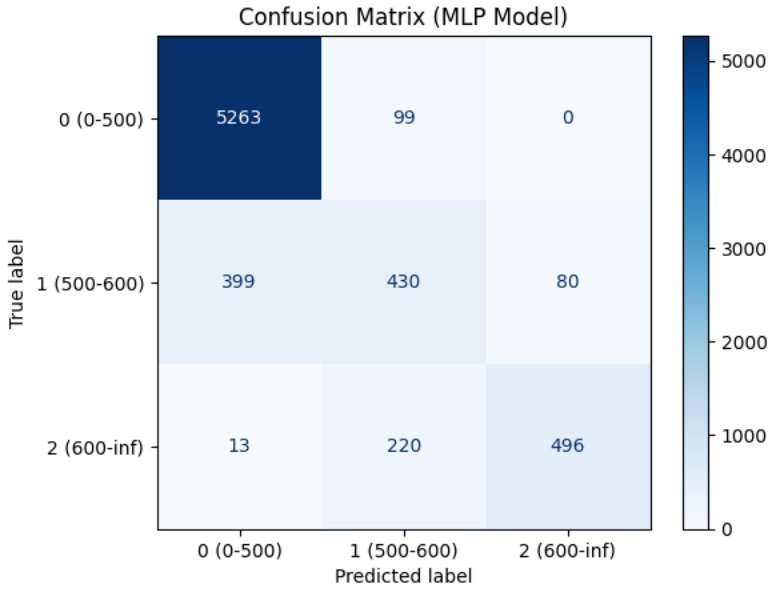
✓ 2m 49.4s

```
Epoch 86/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5790.43it/s]
Epoch 86/100 - Train Loss: 0.0188, Train Acc: 0.8666, Val Loss: 0.0164, Val Acc: 0.8836
Epoch 87/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1152.84it/s]
Epoch 87/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5796.78it/s]
Epoch 87/100 - Train Loss: 0.0189, Train Acc: 0.8640, Val Loss: 0.0159, Val Acc: 0.8884
Epoch 88/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1152.17it/s]
Epoch 88/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5840.14it/s]
Epoch 88/100 - Train Loss: 0.0190, Train Acc: 0.8647, Val Loss: 0.0160, Val Acc: 0.8893
Epoch 89/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1151.80it/s]
Epoch 89/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5767.41it/s]
Epoch 89/100 - Train Loss: 0.0190, Train Acc: 0.8655, Val Loss: 0.0159, Val Acc: 0.8866
Epoch 90/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1097.37it/s]
Epoch 90/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5729.98it/s]
Epoch 90/100 - Train Loss: 0.0188, Train Acc: 0.8652, Val Loss: 0.0159, Val Acc: 0.8887
Epoch 91/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1053.18it/s]
Epoch 91/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5735.70it/s]
Epoch 91/100 - Train Loss: 0.0190, Train Acc: 0.8655, Val Loss: 0.0159, Val Acc: 0.8923
Epoch 92/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1144.35it/s]
Epoch 92/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5767.77it/s]
Epoch 92/100 - Train Loss: 0.0187, Train Acc: 0.8663, Val Loss: 0.0162, Val Acc: 0.8841
Epoch 93/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1148.53it/s]
Epoch 93/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5783.39it/s]
Epoch 93/100 - Train Loss: 0.0188, Train Acc: 0.8653, Val Loss: 0.0164, Val Acc: 0.8819
Epoch 94/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1140.53it/s]
Epoch 94/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5770.00it/s]
Epoch 94/100 - Train Loss: 0.0189, Train Acc: 0.8657, Val Loss: 0.0161, Val Acc: 0.8906
Epoch 95/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1077.02it/s]
Epoch 95/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5335.01it/s]
Epoch 95/100 - Train Loss: 0.0189, Train Acc: 0.8650, Val Loss: 0.0157, Val Acc: 0.8923
Epoch 96/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1092.46it/s]
Epoch 96/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5360.56it/s]
Epoch 96/100 - Train Loss: 0.0187, Train Acc: 0.8643, Val Loss: 0.0159, Val Acc: 0.8847
Epoch 97/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1140.03it/s]
Epoch 97/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5772.00it/s]
Epoch 97/100 - Train Loss: 0.0186, Train Acc: 0.8667, Val Loss: 0.0159, Val Acc: 0.8899
Epoch 98/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1138.91it/s]
Epoch 98/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5808.99it/s]
Epoch 98/100 - Train Loss: 0.0189, Train Acc: 0.8662, Val Loss: 0.0158, Val Acc: 0.8921
Epoch 99/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1146.76it/s]
Epoch 99/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5797.79it/s]
Epoch 99/100 - Train Loss: 0.0186, Train Acc: 0.8672, Val Loss: 0.0159, Val Acc: 0.8903
Epoch 100/100 - Training: 100%|██████████| 1750/1750 [00:01<00:00, 1143.80it/s]
Epoch 100/100 - Validation: 100%|██████████| 438/438 [00:00<00:00, 5771.98it/s]
Epoch 100/100 - Train Loss: 0.0189, Train Acc: 0.8651, Val Loss: 0.0161, Val Acc: 0.8841
```

65010077 กิตติพศ หลำบางช้าง







2. หากทีมงานทดสอบภาคสนามเก็บข้อมูลตัวอย่าง (สมมติคือข้อมูลแถวสุดท้ายที่ลบออกไป) เพื่อต้องการรู้ว่ากังหันลมต่อไปนี้ให้พลังงานเพียงพอที่ควรจะเปิดใช้งานหรือไม่

   ตอบ เพียงพอ เนื่องจาก Predict ได้ 2 แปลว่า จะได้พลังงานมากกว่า 600 kWh

```python
model.eval()
with torch.no_grad():
    outputs = model(X_22_test_tensor)
    y_pred = torch.argmax(outputs, dim=1).cpu().numpy()
    print("Y Pred", y_pred)

    y_true = y_22_test_tensor.cpu().numpy()
    print("Y True", y_true)
```

✓ 0.0s

```
Y Pred [2]
Y True [2]
```

3. จากข้อมูลเดียวกันกับข้อ 2.2 หากแรงลม (wind_speed) ลดลงจนเหลือ 0 เมตร/วินาที พลังงานที่ได้ควรเป็น

   ตอบ พลังงานจะอยู่ในช่วง 0-500 kWh เนื่องจาก model predict ได้ 0

```python
X = df.drop(["energy_output", "turbine_status"], axis=1)[-1:]
X["wind_speed"] = 0
ans_23_test_dataset = torch.tensor(
    X.values,
    dtype=torch.float32,
)
X_23_test_tensor = ans_23_test_dataset
y_23_test_tensor = torch.tensor(df["energy_output"][-1:].values, dtype=torch.long)

model.eval()
with torch.no_grad():
    outputs = model(X_23_test_tensor)
    y_pred = torch.argmax(outputs, dim=1).cpu().numpy()
    print("Y Pred", y_pred)

    # y_true = y_23_test_tensor.cpu().numpy()
    # print("Y True", y_true)
```

✓ 0.0s

```
Y Pred [0]
```